

面向对象三大特征介绍



Python是面向对象的语言，支持面向对象编程的三大特性：继承、封装（隐藏）、多态。

封装（隐藏）

隐藏对象的属性和实现细节，只对外提供必要的方法。相当于将“细节封装起来”，只对外暴露“相关调用方法”。

通过前面学习的“私有属性、私有方法”的方式，实现“封装”。Python追求简洁的语法，没有严格的语法级别的“访问控制符”，更多的是依靠程序员自觉实现。

继承

继承可以让子类具有父类的特性，提高了代码的重用性。

从设计上是一种增量进化，原有父类设计不变的情况下，可以增加新的功能，或者改进已有的算法。

多态

多态是指同一个方法调用由于对象不同会产生不同的行为。生活中这样的例子比比皆是：同样是休息方法，人不同休息方法不同。张三休息是睡觉，李四休息是玩游戏，程序员休息是“敲几行代码”。

实时效果反馈

1. 如下哪一个不是python中面向对象三大特征：

- ☐ A 继承
- ☐ B 封装
- ☐ C 多态
- ☐ D 方法调用

答案

1=>D

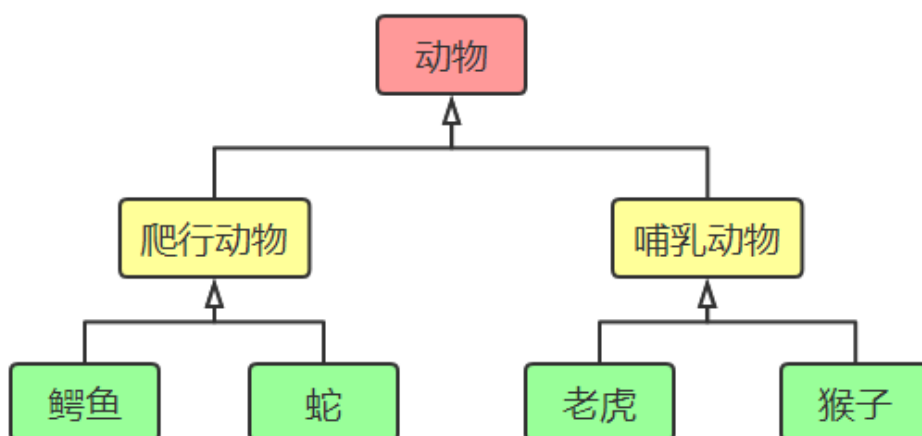
继承

子类扩展父类



继承是面向对象编程的三大特征之一。继承让我们更加容易实现类的扩展。实现代码的重用，不用再重新发明轮子(don't reinvent wheels)。

如果一个新类继承自一个设计好的类，就直接具备了已有类的特征，就大大降低了工作难度。已有的类，我们称为“父类或者基类”，新的类，我们称为“子类或者派生类”。



语法格式

Python支持多重继承，一个子类可以继承多个父类。继承的语法格式如下：

```

1 class 子类类名(父类1[, 父类2, ...]):
2     类体

```

如果在类定义中没有指定父类，则默认父类是 `object` 类。也就是说，`object` 是所有类的父类，里面定义了一些所有类共有的默认实现，比如： `__new__()`

关于构造函数：

- 1 子类不重写 `__init__`，实例化子类时，会自动调用父类定义的 `__init__`。
- 2 子类重写了 `__init__` 时，实例化子类，就不会调用父类已经定义的 `__init__`
- 3 如果重写了 `__init__` 时，要使用父类的构造方法，可以使用 `super` 关键字，也可以使用如下格式调用：

父类名.`__init__`(self, 参数列表)

```

1 class Person:
2     def __init__(self, name, age):
3         print("Person的构造方法")
4         self.name = name
5         self.age = age
6
7     def say_age(self):
8         print(self.name, "的年龄是：", self.age)
9
10
11 class Student(Person):
12
13     def __init__(self, name, age, score):
14         # 子类并不会自动调用父类的__init__()，我们必须显式的调用它。
15         # Person.__init__(self, name, age)
16         #
17         super(Student, self).__init__(name, age)

```

```
17
18         print("Student的构造方法")
19         # self.name = name
20         # self.age = age
21         self.score = score
22
23
24 s1 = Student("张三",15,85)
25 #s1.say_age()
26 print(dir(s1))
```

运行结果：

```
1 张三 的年龄是： 15
2 ['_Person__age', '__class__', '__delattr__',
  '__dict__', '__dir__', '__doc__', '__eq__',
  '__format__', '__ge__', '__getattribute__',
  '__gt__', '__hash__', '__init__',
  '__init_subclass__', '__le__', '__lt__',
  '__module__', '__ne__', '__new__',
  '__reduce__', '__reduce_ex__', '__repr__',
  '__setattr__', '__sizeof__', '__str__',
  '__subclasshook__', '__weakref__', 'name',
  'say_age', 'score']
```

实时效果反馈

1. 如下代码，说法错误的是：

```
1 class Person:
2     def __init__(self, name, age):
```

```
3         self.name = name
4         self.age = age
5
6     def say_age(self):
7         print(self.name, "的年龄是: ", self.age)
8
9
10 class Student(Person):
11
12     def __init__(self, name, age, score):
13         self.score = score
14         Person.__init__(self, name, age)
```

- A** `Person` 的父类是 `object` 类
- B** `Person` 类没有父类
- C** `Student` 类的父类是 `Person` 类, 拥有 `name`、`age`、`score` 属性
- D** `Student` 类的父类是 `Person` 类, 拥有 `say_age` 方法

答案

1=>B

类成员的继承和重写

- ① 成员继承：子类继承了父类除构造方法之外的所有成员。

△(私有属性、私有方法也被继承)

- ② 方法重写：子类可以重新定义父类中的方法，这样就会覆盖父类的方法，也称为“重写”

【操作】继承和重写的案例

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def say_age(self):
7         print(self.name, "的年龄是: ", self.age)
8
9     def say_name(self):
10        print("我是", self.name)
11
12
13 class Student(Person):
14
15     def __init__(self, name, age, score):
16         Person.__init__(self, name, age)
17         self.score = score
18
19     def say_score(self):
20         print(self.name, "的分数
是: ", self.score)
21
22     def say_name(self):          #重写父类的方法
23         print("报告老师, 我是", self.name)
24
25 s1 = Student("张三", 15, 85)
26 s1.say_score()
27 s1.say_name()
28 s1.say_age()
```

执行结果:

```
1 张三 的分数是: 85
2 报告老师, 我是 张三
3 张三 的年龄是: 15
```

查看类的继承层次结构

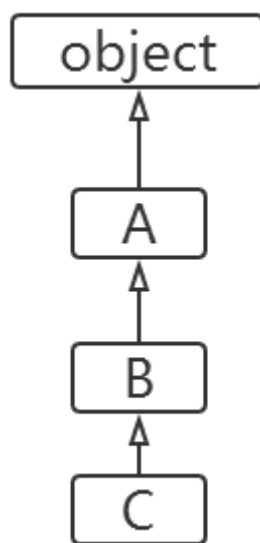
通过类的方法 `mro()` 或者类的属性 `__mro__` 可以输出这个类的继承层次结构。

【操作】 查看类的继承层次结构

```
1 class A:pass
2 class B(A):pass
3 class C(B):pass
4
5 print(C.mro())
```

执行结果:

```
1 [<class '__main__.C'>, <class '__main__.B'>,
   <class '__main__.A'>, <class 'object'>]
```



实时效果反馈

1. 关于Python中类的继承，说法错误的是：

- A** 子类继承了父类除构造方法之外的所有成员
- B** 子类可以重新定义父类中的方法，这样就会覆盖父类的方法，也称为“重写”
- C** 通过类的方法 `mro()` 或者类的属性 `__mro__` 可以输出这个类的继承层次结构
- D** 子类继承了父类的所有成员(包含构造方法)

答案

1=>D

object根类



`object` 类是所有类的父类，因此所有的类都有 `object` 类的属性和方法。我们显然有必要深入研究一下 `object` 类的结构。对于我们继续深入学习Python很有好处。

`dir()` 查看对象属性

为了深入学习对象，先学习内置函数 `dir()`，他可以让我们的看到指定对象所有的属性

【测试】查看对象所有属性以及和 `object` 进行比对

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def say_age(self):
7         print(self.name, "的年龄是: ", self.age)
8
9 obj = object()
10 print(dir(obj))
11
12 s2 = Person("高淇", 18)
13 print(dir(s2))
```

运行结果：

```

1 ['__class__', '__delattr__', '__dir__',
  '__doc__', '__eq__', '__format__', '__ge__',
  '__getattr__', '__gt__', '__hash__',
  '__init__', '__init_subclass__', '__le__',
  '__lt__', '__ne__', '__new__', '__reduce__',
  '__reduce_ex__', '__repr__', '__setattr__',
  '__sizeof__', '__str__', '__subclasshook__']

2
3 ['__class__', '__delattr__', '__dict__',
  '__dir__', '__doc__', '__eq__', '__format__',
  '__ge__', '__getattr__', '__gt__',
  '__hash__', '__init__', '__init_subclass__',
  '__le__', '__lt__', '__module__', '__ne__',
  '__new__', '__reduce__', '__reduce_ex__',
  '__repr__', '__setattr__', '__sizeof__',
  '__str__', '__subclasshook__', '__weakref__',
  'age', 'name', 'say_age']

```

从上面我们可以发现这样几个要点：

- 1 `Person` 对象增加了六个属性：
`__dict__` `__module__` `__weakref__` `age` `name` `say_age`
- 2 `object` 的所有属性，`Person` 类作为 `object` 的子类，显然包含了所有的属性
- 3 我们打印 `age`、`name`、`say_age`，发现 `say_age` 虽然是方法，实际上也是属性。只不过，这个属性的类型是 `method` 而已。

```

1 age <class 'int'>
2 name <class 'str'>
3 say_age <class 'method'>

```

⚠ 关于 `object` 这些属性的详细学习，会后面学习中逐个涉及。在此，没必要展开。

实时效果反馈

1. 如下关于object类，说法错误的是：

- A `object` 类是所有类的父类。
- B 内置函数 `dir()` 可以方便的查看指定对象所有的属性
- C `object` 类，也可以是大写的 `Object` 类
- D 所有的类都有 `object` 类的属性和方法

答案

1=>C

重写 `__str__()` 方法

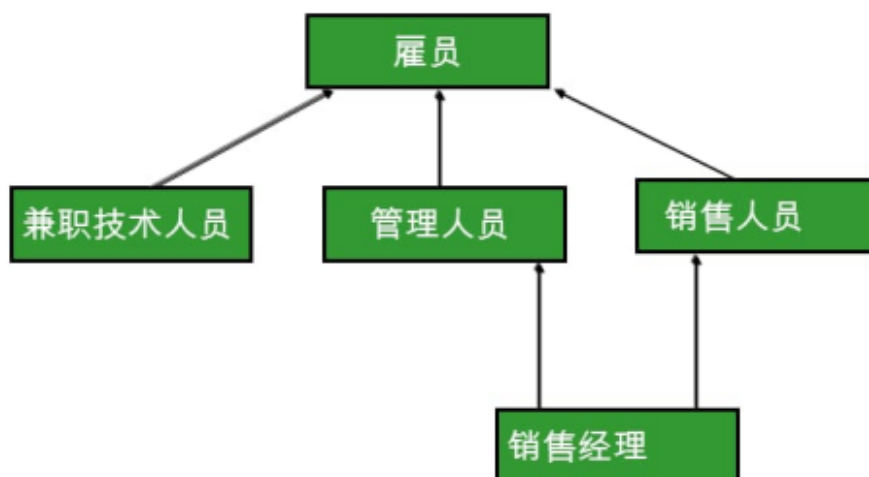
- 1 `object` 有一个 `__str__()` 方法，用于返回一个对于“对象的描述”。内置函数 `str(对象)` ,调用的就是 `__str__()`
- 2 `__str__()` 经常用于 `print()` 方法，帮助我们查看对象的信息。 `__str__()` 可以重写

```

1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.__age = age
5
6     def __str__(self):
7         '''将对象转化成字符串，一般用于print方法'''
8         print("重写__str__方法")
9         return "名字是: {0}, 年龄是
10 {1}".format(self.name, self.__age)
11
12 p = Person("高淇", 18)
13 print(p)
14 s = str(p)

```

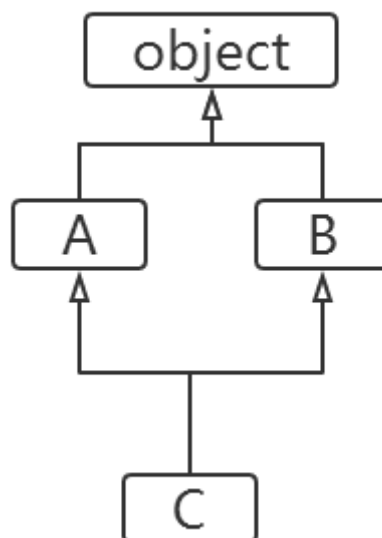
多重继承



Python支持多重继承，一个子类可以有多个“直接父类”。这样，就具备了“多个父类”的特点。但是由于，这样会被“类的整体层次”搞的异常复杂，尽量避免使用。

```
1 class A:
2     def aa(self):
3         print("aa")
4
5 class B:
6     def bb(self):
7         print("bb")
8
9 class C(B,A):
10     def cc(self):
11         print("cc")
12
13 c = C()
14 c.cc()
15 c.bb()
16 c.aa()
```

类结构为：



实时效果反馈

1. 如下关于python中多重继承，说法错误的是：

- A** Python支持多重继承，一个子类可以有多个“直接父类”。
- B** 多重继承可能会让“类的整体层次”搞的异常复杂，尽量不要滥用
- C** `class C(B,A)` 表示：`C` 的父类是 `A` 和 `B`
- D** Python不支持多重继承

答案

1=>D

MRO方法解析顺序

Python支持多继承，如果父类中有相同名字的方法，在子类没有指定父类名时，解释器将“从左向右”按顺序搜索。

MRO (Method Resolution Order)：方法解析顺序。我们可以通过 `mro()` 方法获得“类的层次结构”，方法解析顺序也是按照这个“类的层次结构”寻找的。

```
1 class A:
2     def aa(self):
3         print("aa")
4
5     def say(self):
6         print("say AAA!")
7
8 class B:
9     def bb(self):
```

```

10         print("bb")
11
12     def say(self):
13         print("say BBB!")
14 class C(B,A):
15     def cc(self):
16         print("cc")
17
18 c = C()
19 print(C.mro())           #打印类的层次结构
20 c.say()                  #解释器寻找方法是“从左到
                           #右”的方式寻找，此时会执行B类中的say()

```

执行结果：

```

[<class 'main.C'>, <class 'main.B'>, <class 'main.A'>, <class
'object'>]
say BBB!

```

super()获得父类定义

在子类中，如果想要获得父类的方法时，我们可以通过 `super()` 来做。

`super()` 代表父类的定义，不是父类对象。

♡想调用父类的构造方法：

```
super(子类名称,self).__init__(参数列表)
```

```

1 class A:
2     def __init__(self):

```



```

3         print("A的构造方法")
4
5     def say(self):
6         print("A: ",self)
7         print("say AAA")
8
9 class B(A):
10     def __init__(self):
11         super(B,self).__init__() #调用父类的构造方法
12         print("B的构造方法")
13     def say(self):
14         #A.say(self)      调用父类的say方法
15         super().say()    #通过super()调用父类的方法
16         print("say BBB")
17
18 b = B()
19 b.say()

```

运行结果:

```

1 A:  <__main__.B object at 0x007A5690>
2 say AAA
3 say BBB

```

实时效果反馈

1. 如下关于super(), 说法错误的是:

A `super()` 代表父类的定义, 不是父类对象。

- B** 子类中，如果想要获得父类的方法时，我们可以通过 `super()` 来做
- C** `super()` 代表父类的对象
- D** 用 `super()` 而不是 `super` 代表父类的定义

答案

1=>C

多态



多态 (polymorphism) 是指同一个方法调用由于对象不同可能会产生不同的行为。

比如：现实生活中，同一个方法，具体实现会完全不同。比如：同样是调用人“吃饭”的方法，中国人用筷子吃饭，英国人用刀叉吃饭，印度人用手吃饭。

关于多态要注意以下2点：

- 1 多态是方法的多态，属性没有多态。
- 2 多态的存在有2个必要条件：继承、方法重写

```

1  #多态
2
3  class Animal:
4      def shout(self):

```

```
5         print("动物叫了一声")
6
7     class Dog(Animal):
8         def shout(self):
9             print("小狗，汪汪汪")
10
11    class Cat(Animal):
12
13        def shout(self):
14            print("小猫，喵喵喵")
15
16    def animalShout(a):
17        a.shout()    #传入的对象不同，shout方法对应的
18                    #实际行为也不同。
19
20    animalShout(Dog())
21    animalShout(Cat())
```

实时效果反馈

1. 如下关于多态，说法错误的是：

- A** 多态是指同一个方法调用由于对象不同可能会产生不同的行为
- B** 多态是方法的多态
- C** 多态的存在有2个必要条件：继承、方法重写
- D** 多态是属性的多态

答案

1=>D

特殊方法和运算符重载

Python的运算符实际上是通过调用对象的特殊方法实现的。

```
1 a = 20
2 b = 30
3 c = a+b
4 d = a.__add__(b)
5 print("c=",c)
6 print("d=",d)
```

运算结果：

```
1 c= 50
2 d= 50
```

常见的特殊方法统计如下：

方法	说明	例子
<code>__init__</code>	构造方法	对象创建和初始化： <code>p = Person()</code>
<code>__del__</code>	析构方法	对象回收
<code>__repr__</code> , <code>__str__</code>	打印, 转换	<code>print(a)</code>
<code>__call__</code>	函数调用	<code>a()</code>
<code>__getattr__</code>	点号运算	<code>a.xxx</code>
<code>__setattr__</code>	属性赋值	<code>a.xxx = value</code>
<code>__getitem__</code>	索引运算	<code>a[key]</code>
<code>__setitem__</code>	索引赋值	<code>a[key]=value</code>
<code>__len__</code>	长度	<code>len(a)</code>

每个运算符实际上都对应了相应的方法，统计如下：

运算符	特殊方法	说明
+	<code>__add__</code>	加法
-	<code>__sub__</code>	减法
< <= ==	<code>__lt__</code> <code>__le__</code> <code>__eq__</code>	比较运算符
> >= !=	<code>__gt__</code> <code>__ge__</code> <code>__ne__</code>	比较运算符
^ &	<code>__or__</code> <code>__xor__</code> <code>__and__</code>	或、异或、与
<< >>	<code>__lshift__</code> <code>__rshift__</code>	左移、右移
* / % //	<code>__mul__</code> <code>__truediv__</code> <code>__mod__</code> <code>__floordiv__</code>	乘、浮点除、模运算（取余）、整数除
**	<code>__pow__</code>	指数运算

我们可以重写上面的特殊方法，即实现了“运算符的重载”。

```

1  #测试运算符的重载
2
3  class Person:
4      def __init__(self, name):
5          self.name = name
6
7      def __add__(self, other):
8          if isinstance(other, Person):
9              return "{0}--{1}".format(self.name, other.name)
10         else:
11             return "不是同类对象，不能相加"
12
13     def __mul__(self, other):
14         if isinstance(other, int):
15             return self.name*other
16         else:
17             return "不是同类对象，不能相乘"
18

```

```
19 p1 = Person("高淇")
20 p2 = Person("高希希")
21
22 x = p1 + p2
23 print(x)
24 print(p1*3)
```

运算结果：

```
1 高淇--高希希
2 高淇高淇高淇
```

实时效果反馈

1. 关于运算符和运算符的重载，说法错误的是：

- A** Python的运算符实际上是通过调用对象的特殊方法实现的
- B** 运算符 `+` 对应的特殊方法是： `__add__()`
- C** `__init__` 是构造方法，用于对象初始化
- D** `__del__` 是析构方法，用于对象初始化

答案

1=>D

特殊属性

Python对象中包含了很多双下划线开始和结束的属性，这些是特殊属性，有特殊用法。这里我们列出常见的特殊属性：

特殊属性	含义
<code>obj.__dict__</code>	对象的属性字典
<code>obj.__class__</code>	对象所属的类
<code>class.__bases__</code>	表示类的父类(多继承时, 多个父类放到一个元组中)
<code>class.__base__</code>	类的父类
<code>class.__mro__</code>	类层次结构
<code>class.__subclasses__()</code>	子类列表

```

1  #测试特殊属性
2  class A:
3      pass
4
5  class B:
6      pass
7
8  class C(B,A):
9
10     def __init__(self,nn):
11         self.nn = nn
12
13     def cc(self):
14         print("cc")
15
16 c = C(3)
17
18 print(c.__dict__)
19 print(c.__class__)
20 print(C.__bases__)
21 print(C.mro())
22 print(A.__subclasses__())

```

运行结果:

```

1  ['__class__', '__delattr__', '__dict__',
   '__dir__', '__doc__', '__eq__', '__format__',
   '__ge__', '__getattribute__', '__gt__',
   '__hash__', '__init__', '__init_subclass__',
   '__le__', '__lt__', '__module__', '__ne__',
   '__new__', '__reduce__', '__reduce_ex__',
   '__repr__', '__setattr__', '__sizeof__',
   '__str__', '__subclasshook__', '__weakref__',
   'cc', 'nn']
2  {'nn': 3}
3  <class '__main__.C'>
4  (<class '__main__.B'>, <class '__main__.A'>)
5  [<class '__main__.C'>, <class '__main__.B'>,
   <class '__main__.A'>, <class 'object'>]
6  [<class '__main__.C'>]

```

实时效果反馈

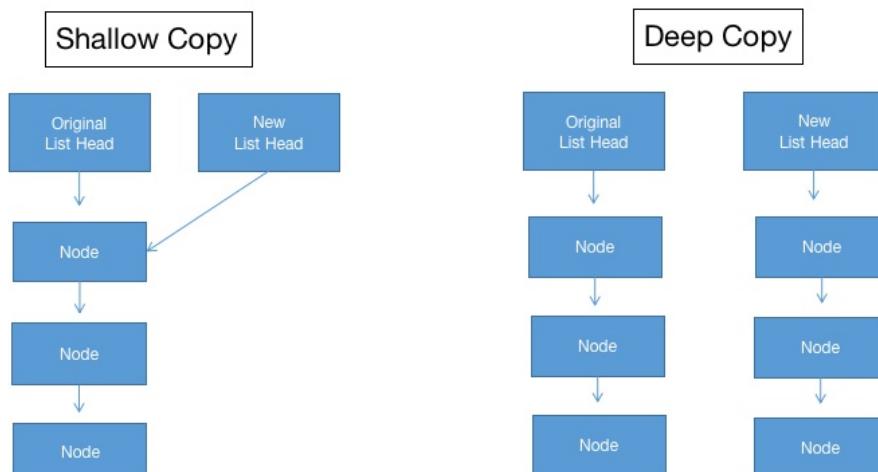
1. Python对象中有很多双下划线开始和结束的特殊属性，说法错误的是：

- A** `obj.__dict__` 是对象的属性字典
- B** `obj.__class__` 表示对象所属的类的层次结构
- C** `class.__bases__` 表示类的父类(多继承时，多个父类放到一个元组中)
- D** `class.__mro__` 表示类的层次结构

答案

1=>B

对象的浅拷贝和深拷贝



浅拷贝

Python拷贝一般都是浅拷贝。

浅拷贝：拷贝时，拷贝源对象，但对象包含的子对象内容不拷贝。

深拷贝

使用 `copy` 模块的 `deepcopy` 函数，递归拷贝对象中包含的子对象。

深拷贝：拷贝时，拷贝源对象，也递归拷贝对象中包含的子对象

```

1  #测试对象的引用赋值、浅拷贝、深拷贝
2  import copy
3
4  class MobilePhone:
5      def __init__(self,cpu):
6          self.cpu = cpu
7
8  class CPU:
9      pass
10

```

```

11 c = CPU()
12 m = MobilePhone(c)
13
14 print("----浅拷贝-----")
15 m2 = copy.copy(m)    #m2是新拷贝的另一个手机对象
16 print("m: ",id(m))
17 print("m2: ",id(m2))
18 print("m的cpu: ",id(m.cpu))
19 print("m2的cpu: ",id(m2.cpu))    #m2和m拥有了一
    样的cpu对象
20 print("----深拷贝-----")
21 m3 = copy.deepcopy(m)
22 print("m: ",id(m))
23 print("m3: ",id(m3))
24 print("m的cpu: ",id(m.cpu))
25 print("m3的cpu: ",id(m3.cpu))    #m3和m拥有不一
    样的cpu对象

```

运算结果:

```

1  ----浅拷贝-----
2  m:  1879267229360
3  m2:  1879267228592
4  m的cpu:  1879267229648
5  m2的cpu:  1879267229648
6  ----深拷贝-----
7  m:  1879267229360
8  m3:  1879267222256
9  m的cpu:  1879267229648
10 m3的cpu:  1879267221968

```

实时效果反馈

1. 关于浅拷贝和深拷贝，说法错误的是：

- A** Python拷贝一般都是浅拷贝。
- B** 深拷贝：拷贝时，拷贝源对象，也递归拷贝对象中包含的子对象。
- C** 深拷贝：拷贝时，拷贝源对象，也拷贝一次子对象
- D** 浅拷贝：拷贝时，拷贝源对象，但对象包含的子对象内容不拷贝。

答案

1=>C

组合



结婚就是组合。两人组合后，可以复用对方的属性和方法！

除了继承，“组合”也能实现代码的复用！“组合”核心是“将父类对象作为子类的属性”。

- 1 **is-a** 关系，我们可以使用“继承”。从而实现子类拥有的父类的方法和属性。**is-a** 关系指的是类似这样的关系：狗是动物，dog is animal。狗类就应该继承动物类。
- 2 **has-a** 关系，我们可以使用“组合”，也能实现一个类拥有另一个类的方法和属性。**has-a** 关系指的是这样的关系：手机拥有CPU。MobilePhone has a CPU

```
1  #组合测试
2
3  class MobilePhone:
4      def __init__(self,cpu,screen):
5          self.cpu = cpu
6          self.screen = screen
7
8
9  class CPU:
10
11      def calculate(self):
12          print("计算，算个12345")
13
14
15  class Screen:
16      def show(self):
17          print("显示一个好看的画面，亮瞎你的钛合金大眼")
18
19
20  c = CPU()
21  s = Screen()
22  m = MobilePhone(c,s)
23  m.cpu.calculate()          #通过组合，我们也能调用cpu对象的方法。相当于手机对象间接拥有了“cpu的方法”
```

```
24 m.screen.show()
```

运算结果：

- 1 计算，算个12345
- 2 显示一个好看的画面，亮瞎你的钛合金大眼

实时效果反馈

1. 如下关于组合和继承，说法错误的是：

- A** 继承可以实现代码复用；组合不能实现代码复用。
- B** 组合的核心是“将父类对象作为子类的属性”
- C** `is-a` 关系，我们可以使用“继承”
- D** `has-a` 关系，我们可以使用“组合”

答案

1=>A

设计模式_工厂模式实现



世界~~残酷~~，充满套路
美好

设计模式是面向对象语言特有的内容，是我们在面临某一类问题时候固定的做法，设计模式有很多种，比较流行的是：GOF (Group Of Four) 23种设计模式。当然，我们没有必要全部学习，学习几个常用的即可。

对于初学者，我们学习两个最常用的模式：工厂模式和单例模式。

工厂模式实现了创建者和调用者的分离，使用专门的工厂类将选择实现类、创建对象进行统一的管理和控制。

工厂模式

创建者和调用者的分离



```
1  #工厂模式
2
3  class CarFactory:
4      def createCar(self, brand):
5          if brand == "奔驰":
6              return Benz()
7          elif brand == "宝马":
8              return BMW()
9          elif brand == '比亚迪':
10             return BYD()
11         else:
12             return "未知品牌，无法创建"
13
14
15 class Benz:
16     pass
17
18
19 class BMW:
20     pass
21
22
23 class BYD:
24     pass
25
26 factory = CarFactory()
27 c1 = factory.createCar("奔驰")
28 c2 = factory.createCar("宝马")
29 print(c1)
30 print(c2)
```

运行结果：

```
1 <__main__.Benz object at 0x021c5770>  
2 <__main__.BMW object at 0x021c5790>
```

设计模式_单例模式实现



单例模式 (Singleton Pattern) 的核心作用是确保一个类只有一个实例，并且提供一个访问该实例的全局访问点。

单例模式只生成一个实例对象，减少了对系统资源的开销。当一个对象的产生需要比较多的资源，如读取配置文件、产生其他依赖对象时，可以产生一个“单例对象”，然后永久驻留内存中，从而极大的降低开销。

⚠ 单例模式有多种实现的方式，我们这里推荐重写 `__new__()` 的方法。

```
1 #单例模式  
2  
3 class MySingleton:  
4     __obj = None  
5     __init_flag = True  
6  
7     def __new__(cls, *args, **kwargs):  
8         if cls.__obj == None:
```



```
9         cls.__obj = object.__new__(cls)
10
11     return cls.__obj
12
13     def __init__(self, name):
14         if MySingleton.__init_flag:
15             print("init....")
16             self.name = name
17             MySingleton.__init_flag = False
18
19 a = MySingleton("aa")
20 print(a)
21 b = MySingleton("bb")
22 print(b)
```

运算结果:

```
1 init....
2 <__main__.MySingleton object at 0x01E15610>
3 <__main__.MySingleton object at 0x01E15610>
```

工厂和单例模式结合

自古深情留不住 总是套路得人心



设计模式称之为“模式”，就是一些固定的套路。我们很容易用到其他场景上，比如前面讲的工厂模式，我们需要将工厂类定义成“单例”，只需要简单的套用即可实现：

```
1  #测试工厂模式和单例模式的整合使用
2  class CarFactory:
3      __obj = None          #类属性
4      __init_flag = True
5
6
7      def create_car(self, brand):
8          if brand == "奔驰":
9              return Benz()
10         elif brand == "宝马":
11             return BMW()
12         elif brand == "比亚迪":
13             return BYD()
14         else:
15             return "未知品牌，无法创建"
16
17
18     def __new__(cls, *args, **kwargs):
```

```
19         if cls.__obj ==None:
20             cls.__obj = object.__new__(cls)
21
22         return cls.__obj
23
24     def __init__(self):
25         if CarFactory.__init_flag:
26             print("init CarFactory....")
27             CarFactory.__init_flag = False
28
29 class Benz:
30     pass
31
32 class BMW:
33     pass
34
35 class BYD:
36     pass
37
38 factory = CarFactory()
39 c1 = factory.create_car("奔驰")
40 c2 = factory.create_car("比亚迪")
41 print(c1)
42 print(c2)
43
44 factory2 = CarFactory()
45 print(factory)
46 print(factory2)
```

运算结果：

```

1 init CarFactory....
2 <__main__.Benz object at 0x01E36E90>
3 <__main__.BYD object at 0x01E36C30>
4 <__main__.CarFactory object at 0x01E36730>
5 <__main__.CarFactory object at 0x01E36730>

```

实操作业

- ① 如下代码测试对象的浅拷贝、深拷贝，请绘制出内存示意图。

```

1 #测试对象的引用赋值、浅拷贝、深拷贝
2 import copy
3
4 class MobilePhone:
5     def __init__(self,cpu,screen):
6         self.cpu = cpu
7         self.screen = screen
8
9
10 class CPU:
11
12     def calculate(self):
13         print("计算，算个12345")
14         print("CPU对象:",self)
15
16 class Screen:
17     def show(self):
18         print("显示一个好看的画面，亮瞎你的钛合金大眼")
19         print("屏幕对象: ",self)
20
21

```

```

22 c = CPU()
23 s = Screen()
24 m = MobilePhone(c,s)
25
26 m.cpu.calculate()
27
28 n = m                                #两个变量，但是指向了同一个对象
29 print(m,n)
30
31 m2 = copy.copy(m)                    #m2是新拷贝的另一个手机对象
32 print(m,m2)
33
34 m.cpu.calculate()
35 m2.cpu.calculate()                  #m2和m拥有了一样的cpu对象和screen对象
36
37 m3 = copy.deepcopy(m)
38 m3.cpu.calculate()                  #m3和m拥有不一样的cpu对象和screen对象

```

- ② 定义发动机类Motor、底盘类Chassis、座椅类Seat，车辆外壳类Shell，并使用组合关系定义汽车类。其他要求如下：

定义汽车的run()方法，里面需要调用Motor类的工作()方法，也需要调用座椅类Seat的工作()方法，也需要调用底盘类Chassis的工作()方法

- ③ 使用工厂模式、单例模式实现如下需求：

- (1) 电脑工厂类ComputerFactory用于生产电脑Computer。工厂类使用单例模式，也就是说只能有一个工厂对象。
- (2) 工厂类中可以生产各种品牌的电脑：联想、华硕、神舟

- (3) 各种品牌的电脑使用继承实现：
 - (4) 父类是Computer类，定义了calculate方法
 - (5) 各品牌电脑类需要重写父类的calculate方法
- 4 定义一个Employee雇员类，要求如下：
- (1) 属性有：id、name、salary
 - (2) 运算符重载+：实现两个对象相加时，默认返回他们的薪水和
 - (3) 构造方法要求：输入name、salary，不输入id。id采用自增的方式，从1000开始自增，第一个新增对象是1001，第二个新增对象是1002。
 - (4) 根据salary属性，使用@property设置属性的get和set方法。set方法要求输入：1000-50000范围的数字。